FinGuide: An Automated Platform for Israeli Payslip Analysis and Personalized Financial Guidance

by

Shahar Mayster

Ofek Dil

Segev Partush

Ofir Raz

Emily Belensky

Approved by the supervisor: Eliav Menashe

Submitted to the Computer Science Faculty of College of Management

July 2026, Rishon LeZion

2

Acknowledgments

We would like to thank the Computer Science Faculty of the College of Management for the academic environment, project-day structure, and support throughout our degree. We are especially grateful to our supervisor, Eliav Menashe, for his guidance, technical feedback, and availability during the research and implementation phases.

We thank the open-source communities behind Tesseract OCR, Node.js, React, and MongoDB, whose tools formed the technical foundation of FinGuide. Finally, we thank our families and teammates for their patience and encouragement during the development of this work.

3

Executive Summary

Israeli salaried employees receive monthly payslips (תלושי שכר) that encode salary, statutory deductions, and employer contributions, yet most lack tools to verify compliance or act on the data. Employers and payroll vendors issue complex documents governed by income tax, National Insurance, and mandatory pension law; employees rarely have the expertise or software to detect missing deposits, rate inconsistencies, or longitudinal gaps. FinGuide addresses this gap with a full-stack web platform that automates payslip ingestion, Hebrew-aware extraction, compliance checking, and personalized financial guidance.

The system applies a multi-path PDF and OCR pipeline, normalizes results into a canonical `analysisData` schema (version 1.9), and runs rule-based detectors for missing pension deposits, contribution-rate gaps, and deposit continuity breaks. A hybrid AI assistant combines deterministic intent routing with Claude and Ollama fallback; a multi-agent layer synthesizes payslip, pension, insurance, and profile analysis into a financial health score and prioritized action items. The implementation uses a Node.js and Express backend with a React 19 and TypeScript frontend, connected through nineteen REST route modules.

Development followed an iterative, backend-first methodology with golden-fixture regression tests, reproducible `eval:*` scripts, and Jest-based unit and integration testing (109 backend and 3 frontend test files). Evaluation on a seven-fixture OCR corpus reported perfect accuracy on core salary fields; a seven-scenario findings set achieved 100% precision and recall; a thirty-nine-query Hebrew routing set reached 94.9% intent classification accuracy; and Path-1 extraction latency on the same corpus measured a 12 ms median on development hardware. FinGuide demonstrates that employee-facing, Hebrew-native payslip analysis is feasible and establishes a foundation for expanded regulatory coverage and advisory features.

4

Table of Contents

5

6

Table of Abbreviations

Abbreviation	Expansion
API	Application Programming Interface
CSS	Cascading Style Sheets
DPI	Dots Per Inch
ESM	ECMAScript Module
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
ILS	Israeli New Shekel (₪)
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MVC	Model–View–Controller
NII	National Insurance Institute (ביטוח לאומי)
NLP	Natural Language Processing
OCR	Optical Character Recognition
OEM	OCR Engine Mode
PDF	Portable Document Format
PNG	Portable Network Graphics
PSM	Page Segmentation Mode
REST	Representational State Transfer
RTL	Right-to-Left
SPA	Single-Page Application
SQL	Structured Query Language
SSE	Server-Sent Events

7

Abbreviation	Expansion
UUID	Universally Unique Identifier

8

Table of Figures

9

Table of Tables

10

Chapter 1: Introduction

This chapter introduces the problem FinGuide addresses, the objectives and scope of the project, and the methodology used to carry out the work. It establishes the context and significance of Israeli payslip analysis and outlines how the remainder of this project book is organized.

1.1 Background

The management of personal finances is a complex and deeply consequential activity that affects every employed individual. For Israeli salaried workers, this complexity is amplified by the structure of the local labor market and the regulatory environment. Every employee in Israel receives a monthly payslip (תלוש שכר) that documents not only their gross and net salary but also a web of statutory deductions and employer contributions governed by multiple legislative frameworks: the Income Tax Ordinance (פקודת מס הכנסה), the National Insurance Law (חוק הביטוח הלאומי), the Pension Obligation Law (חוק הפנסיה החובה, 2008), and the Individual Labor Agreements and Collective Bargaining Agreements that set the terms for study fund (קרן השתלמות) participation.

A typical Israeli payslip includes dozens of distinct line items. Gross salary may be composed of base salary, travel allowance, meal allowance, commissions, overtime premiums, and other components. From this gross amount, the employer deducts income tax (מס הכנסה), the employee's share of National Insurance (ביטוח לאומי), and the employee's health insurance levy (מס בריאות). Simultaneously, the employer transfers contributions to the employee's pension fund (קרן פנסיה) or provident fund (קופת גמל) — an amount that includes the employee's own contribution (typically 6% of pensionable salary), the employer's contribution (typically 6.5%), and a severance provision (typically 6%) — as well as employer and employee contributions to a study fund when applicable. The result is a document that functions as both a wage statement and a compliance certificate for multiple overlapping regulatory obligations.

Despite this complexity, most Israeli employees have limited means to analyze their payslips beyond reading the net payable figure at the bottom. Commercial accounting software exists for businesses but is rarely targeted at individual employees seeking to audit their own payslips. Generic personal finance applications lack the domain-specific models required to interpret Israeli payslip structure, and the language barrier created by Hebrew text further limits the utility of international tools. This situation creates a systematic blind spot: employees may be unknowingly receiving below-minimum pension contributions, ex-

11

periencing unnoticed deposit gaps, or failing to identify discrepancies between stated and implied contribution rates — all of which can translate into material financial harm over time.

FinGuide was developed to close this gap. It provides automated payslip ingestion, structured financial data extraction, statutory compliance verification, personalized financial guidance, and longitudinal analysis of payslip history — all within a Hebrew-language, mobile-responsive web interface accessible to employees with no specialist financial knowledge.

1.2 Problem Statement

The problem addressed by this project may be stated as follows: Israeli salaried employees receive complex monthly payslips containing critical financial and regulatory information that the overwhelming majority are unable to independently verify, analyze, or act upon. Three specific failure modes motivate this work:

Missing or incomplete pension deposits (addressed by Objective 2). Under Israeli law, pension participation is mandatory for most salaried employees since 2008. However, employers may fail to allocate contributions for new employees during a probationary period, for employees working below minimum thresholds, or due to administrative errors. Without automated detection, such failures may go unnoticed for months or years.

Contribution rate gaps (addressed by Objective 2). Even when pension or study fund contributions are recorded on the payslip, the stated rate (as a percentage) and the implied rate (derived from the deposited amount divided by the applicable salary base) may diverge. Such gaps can indicate calculation errors, incorrect salary base definitions, or deliberate under-contribution. Manual cross-checking requires spreadsheet calculations that most employees do not perform.

Temporal continuity gaps (addressed by Objective 2). An employee's total pension accumulation depends on uninterrupted contribution for the duration of their working career. Gaps — months in which no contribution appears on the payslip — reduce the terminal balance. Detecting such gaps requires comparing payslip history over time, a task impractical without an automated system.

Beyond these three compliance-oriented problems, there is a broader challenge of financial literacy and actionability (addressed by Objectives 3, 4, and 5). Even employees who understand their payslip in isolation often lack the context to evaluate whether their salary tra-

12

jectory is normal, whether their deductions are correctly computed, or whether they are on track to meet retirement savings targets.

1.3 Objectives

The primary objectives of this project are:

1. **Automated payslip ingestion and OCR.** Design and implement a document processing pipeline that reliably extracts structured financial data from Israeli payslip PDFs, handling Hebrew text, varied layouts, scanned documents, and password-protected files.

1. **Financial findings detection.** Build a rule-based analysis engine that detects missing pension or study fund deposits, contribution rate inconsistencies, and deposit continuity gaps, with findings linked to specific payslip periods.

1. **Personalized AI assistant.** Integrate a hybrid language model system that answers financial queries in Hebrew using deterministic rule-based responses for well-defined intents and LLM inference for open-ended questions, grounded in the user's actual payslip data.

1. **Multi-agent financial analysis.** Design and implement a multi-agent orchestration architecture that runs specialized domain analyzers in parallel and synthesizes their outputs into an aggregate financial health score and actionable recommendations.

1. **Longitudinal financial planning.** Implement a savings forecast module that projects pension accumulation over the user's career horizon and supports scenario-based what-if analysis.

1. **Accessible, Hebrew-native UI.** Build a full right-to-left web interface in React that requires no financial expertise to navigate, surfaces findings prominently, and integrates the AI assistant as a conversational layer over the user's financial data.

1.4 Scope and Limitations

In scope: The system processes Israeli salary payslips (תלושי שכר) in PDF format from standard Hebrew-language payroll exports. Regulatory models reflect Israeli law as of 2026, including prevailing income tax brackets, National Insurance rates, pension minimum contribution rates, and study fund thresholds. The system supports one to approximately fifty payslip documents per user and stores data in a MongoDB instance.

13

Out of scope: The system does not provide certified financial advice; all outputs are informational and disclaimed accordingly. It does not file taxes, interact with pension fund management companies directly, or support the full range of Form 106 (annual income report) processing beyond basic metadata extraction. Support for languages other than Hebrew and English is not implemented. The savings forecast model is a linear projection that does not account for investment returns or inflation. Integration with banking APIs, pension fund portals, or the Israeli Tax Authority API is not part of the current implementation but is discussed as future work.

Product limitations: OCR accuracy depends on source PDF quality and layout consistency; heavily scanned or non-standard payslips may require manual field entry via `needs_review` status. The LLM assistant may cite outdated regulatory figures unless overridden by system-prompt constants; users are advised to verify specific statutory amounts.

1.5 Methodology

The project was developed using an iterative, feature-driven approach broadly aligned with the Agile methodology. Work was organized into thematic verticals — document ingestion, findings detection, AI assistant, multi-agent analysis, and frontend — which were developed and tested independently before integration. Each feature was validated against a corpus of real payslips obtained with user consent, supplemented by synthetic documents generated for testing edge cases.

Development timeline. The project proceeded in four phases aligned with repository history:

1. **Phase 1 — Backend foundation (months 1–3).** Definition of the `analysisData` v1.9 schema, document upload API, authentication, and the multi-path OCR pipeline (`payslipOcr.js`, golden fixtures). Findings detectors and statutory threshold configuration.
2. **Phase 2 — Core product (months 4–6).** React frontend with RTL layout, payslip history, findings UI, and `documentToPayslip.ts` mapping layer. Integration of insights, recommendations, and savings forecast.
3. **Phase 3 — AI layer (months 7–9).** Hybrid assistant (`detectIntent`, `claudeChatService`), multi-agent orchestration (`full-analysis`), financial health score, and extended domains (pension, insurance, copilot, Gmail import).

14

4. **Phase 4 — Evaluation and documentation (months 10–12).** Reproducible `eval:ocr`, `eval:findings`, and `eval:ai-routing` scripts; Jest regression suite; project book and submission packaging.

The backend was developed first, as it defines the data contracts on which all other functionality depends. The `analysisData` canonical schema was defined early and treated as a stable interface; all OCR improvements, golden-fixture regression tests, and manual field-filling flows were constrained to produce outputs conforming to this schema. The frontend was developed in parallel after the core API endpoints were stabilized, using mock responses during the period before backend integration was complete.

Testing followed a mixed strategy: unit tests verified the correctness of isolated financial calculations (contribution rate gap detection, deposit continuity timeline construction, savings forecast arithmetic); integration tests verified the full request-to-response cycle including database writes; and manual exploratory testing evaluated OCR quality on real documents. The full test suite is executed via Jest with the `--runInBand` flag to prevent database contention.

Reproducible evaluation harnesses complement manual testing: `npm run eval:ocr` scores field extraction against a golden fixture corpus; `npm run eval:findings` and `npm run eval:ai-routing` measure findings detection and intent classification on annotated scenario sets. Results from these scripts are reported in Chapter 4.

1.6 Organization of the Project Book

Chapter 2 reviews the relevant academic and technical literature. Chapter 3 presents system architecture, implementation details, and evaluation metrics. Chapter 4 reports experimental setup, results, analysis, and comparison with existing approaches. Chapter 5 concludes with contributions, limitations, and future work. The front matter includes tables of abbreviations, figures, and tables, plus references; appendices provide API documentation, setup instructions, and schema reference.

1.7 Team Contributions

Contributions reflect primary ownership areas verified against repository commit history:

15

Team member	Primary contributions
Shahar Mayster	Project coordination, findings utilities (`detectContributionRateGap` , `detectDepositContinuityGap`), evaluation scripts, project book authoring, integration testing
Ofek Dil	Frontend architecture (React/Vite), Hub and payslip UI, OCR service integration, primary UI development
Ofir Raz	Backend controllers and API layer, AI assistant wiring, document processing, authentication flows
Emily Belensky	Backend services (OCR pipeline, financial document processing), frontend components, test coverage
Segev Partush	Multi-agent orchestration, pension and insurance domains, AI prompt and agent tooling

All team members participated in code review, manual payslip validation, and end-to-end testing of the integrated platform.

16

Chapter 2: Literature Review

This chapter surveys the academic and technical literature relevant to FinGuide. Sections 2.2 through 2.7 examine document digitization, Hebrew language processing, Israeli employment regulation, personal financial management systems, web architecture, and AI in fintech. Section 2.8 synthesizes these strands and states the research gap that motivates the project.

2.1 Overview of Relevant Literature

FinGuide sits at the intersection of four research areas: document digitization and OCR, natural language processing for Hebrew, computational models of personal finance grounded in Israeli labor law, and AI-assisted advisory systems. The subsections below treat each area in depth; Section 2.8 consolidates how they inform the system's design choices and identifies what prior work does not provide.

2.2 Document Digitization and Optical Character Recognition

2.2.1 Text-Layer Extraction versus Image-Based OCR

Modern payroll PDFs are often generated programmatically and embed a searchable text layer. Direct extraction via tools such as Poppler's `pdftotext` is faster and typically more accurate than image-based OCR when the text layer is intact [1]. However, Israeli payroll exports sometimes produce encoding artifacts — Hebrew Unicode characters replaced by replacement glyphs (U+FFFD) — which forces a fallback to rasterization and OCR [3]. The extraction pipeline must therefore support both paths and select among them based on measurable text quality rather than file format alone.

2.2.2 OCR Engines and Page Segmentation

The Tesseract engine, originally developed at Hewlett-Packard and later open-sourced under Google, remains the dominant open-source OCR tool [1]. Version 4.0 introduced an LSTM-based neural network architecture (OEM 1) that substantially improved recognition on complex scripts compared to the legacy character-pattern classifier (OEM 0) [2]. Tesseract supports Hebrew (`heb`) and provides several page segmentation modes (PSM): PSM 6 treats the image as a single uniform text block; PSM 4 assumes a single column of variable-size text; PSM 3 performs fully automatic segmentation. Research on document layout analysis shows that no single PSM mode is optimal across all document types, motivating multi-candidate evaluation and ranking [3].

17

2.2.3 Preprocessing and Evaluation Challenges

Image preprocessing prior to OCR significantly affects recognition quality. Converting to grayscale, normalizing contrast, and applying threshold binarization improve character recognition on both printed and scanned documents [4]. For uniformly illuminated printer-generated payslips, a fixed global threshold is often sufficient and avoids the computational cost of adaptive methods. Evaluation of extraction systems is complicated by semi-structured layouts: accuracy must be measured at the field level (gross salary, net payable, contribution amounts) rather than by character error rate alone, because downstream compliance checking depends on correctly attributed numeric values [6].

2.2.4 Mixed-Script and Bidirectional Text

Israeli payslips typically mix Hebrew labels, Latin-script identifiers, Arabic numerals, and currency symbols. This requires a combined language model (`heb+eng`) and careful handling of bidirectional text, where reading order alternates between right-to-left (Hebrew) and left-to-right (numbers and Latin tokens) within a single line [5], [18]. Field extraction therefore relies on domain-specific label dictionaries and positional heuristics rather than generic sentence-level NLP.

2.3 Hebrew Language Processing

Hebrew presents challenges beyond those of Latin-script languages. The writing system is a consonantal alphabet (abjad) in which vowel markings (nikud) are optional and absent in professional payslips. Ambiguity at the character level propagates to word level in ways that do not occur in fully vocalized scripts [5]. Hebrew is also morphologically rich: a single stem may produce dozens of inflected forms through prefixation and suffixation.

For payslip field extraction, however, full morphological analysis is not required. The vocabulary of Israeli payslip labels is finite and domain-specific — terms such as "ברוטו", "נטו", "לתשלום", and "ביטוח לאומי" recur across vendors with limited variation. Keyword-based matching with normalization suffices for label identification in this constrained setting [6]. Research on multilingual information extraction from semi-structured documents has examined approaches from rule-based template matching to transformer-based layout models such as LayoutLM [6]. For high-stakes financial documents, rule-based approaches offer determinism, explainability, and graceful degradation: when a field is not found, the system returns null rather than a confabulated value.

18

2.4 Israeli Financial Regulations Governing Employment

The financial calculations performed in employee-facing compliance tools are grounded in Israeli labor and social security law. The key regulatory frameworks are:

The Income Tax Ordinance (פקודת מס הכנסה). Income tax in Israel is levied at progressive marginal rates. For the **2026 statutory floors** published by the Israel Tax Authority, the bracket structure begins at 10% for income up to approximately ₪84,120 annually, rising through five additional brackets to 50% for income exceeding ₪876,720 annually [14]. Employees are entitled to credit points (נקודות זיכוי); each credit point reduces annual tax liability by a fixed shekel amount set annually by the Tax Authority [14].

National Insurance Law (חוק הביטוח הלאומי). The National Insurance Institute (ביטוח לאומי) collects compulsory contributions from both employee and employer [15]. Employee contributions fund old-age, disability, survivors', and unemployment benefits, together with a health levy. Employer NII contributions are paid in addition to the employee's deduction. FinGuide's deduction validators compare extracted amounts against NII rate tables for the active tax year [15].

Pension Obligation Law (2008 ,חוק הפנסיה החובה) and subsequent amendments. Since 2008, pension participation has been mandatory for most salaried employees in Israel with limited exceptions [7]. Minimum contribution rates (2026 statutory floors, as a percentage of pensionable salary): employee 6%, employer contribution 6.5%, and employer severance provision 6% (or a combined contribution-and-severance model at 12.5%) [7]. Collective bargaining agreements and individual contracts frequently specify higher rates.

Study Fund (קרן השתלמות). Study funds are a tax-advantaged savings vehicle. Employer contributions of up to 7.5% of salary and employee contributions of up to 2.5% may receive tax exemptions up to statutory caps [14]. Participation is not universally mandatory but is common under collective agreements.

Compliance checking maps finding types to legal bases: *missing deposit* → Pension Obligation Law minimum allocation [7]; *rate gap* → stated vs implied percent against statutory floors in `contributionRateThresholds.js` [7]; *continuity gap* → uninterrupted pensionable service expectation under pension accumulation practice [7], [15].

19

2.5 Personal Financial Management Systems

The personal financial management (PFM) category spans budget trackers to wealth management platforms. Academic surveys identify recurrent design tensions: comprehensive data collection (requiring account aggregation) versus privacy; automation versus user agency; and moving users from passive awareness to behavior change [8].

Israeli employees face an additional obstacle. International PFM tools such as Mint, YNAB, and Copilot support bank aggregation in major English-speaking markets, but no equivalent with Israeli bank integration and Hebrew-native payslip parsing was available when this project was initiated [16]. Employer-integrated payslip portals and bank apps offer partial coverage but not employee-centric compliance analysis (see Table in §2.5.1).

A payslip-centric approach — accepting the PDF as primary input rather than banking credentials — reduces privacy risk while targeting the document that encodes regulated employer obligations [8], [17].

2.5.1 Employer Portals versus Employee-Centric Tools

Capability	**Hilan / iCount portal**	**Bank PFM app**	**FinGuide**
Structured payslip display	Yes (employer-scoped)	No (transactions only)	Yes (user-uploaded)
Cross-employer history	No	Partial (bank view)	Yes
Compliance findings (deposit/rate/continuity)	No	No	Yes
Hebrew-native advisory	No	Limited	Yes (rule + LLM)
Banking API required	No	Yes	No

Employer portals excel as authoritative payslip viewers tied to payroll systems; bank apps excel at cash-flow visibility [16]. Neither performs statutory minimum checking against payslip line items — the gap FinGuide targets [8].

20

2.6 Web Application Architecture Patterns

Consumer-facing financial applications commonly use a REST API backend with a single-page application frontend [9]. Layered backend architectures separate routes, controllers, services, and persistence, facilitating unit testing and localizing the impact of changes. Node.js and Express provide non-blocking I/O suited to I/O-intensive workloads: file reads, OCR subprocess execution, database operations, and external API calls.

Document-oriented databases such as MongoDB suit evolving extraction schemas: when the extraction algorithm improves, new fields are added without relational migrations, at the cost of reduced query expressiveness on nested fields [9]. The frontend pattern — React with TypeScript, compiled by Vite, with a dedicated mapping layer at the API boundary — provides compile-time type safety while allowing the backend `analysisData` contract to evolve via a `schema_version` field. Right-to-left layout is handled through CSS `direction: rtl` at the document root, with component-level overrides for Latin and numeric content.

2.7 AI and Large Language Models in Fintech

The integration of large language models into financial advisory applications has attracted significant attention since instruction-tuned models demonstrated strong few-shot performance [10]. LLMs perform well on financial reasoning benchmarks but exhibit confabulation of specific numerical facts — a serious failure mode when quoting tax rates or pension contribution thresholds [11].

Hybrid architectures address this risk by routing structured, rule-sensitive queries to deterministic handlers grounded in user data, reserving LLM inference for open-ended questions [11]. The multi-agent paradigm — specialized agents for distinct analytical domains with an orchestrator synthesizing outputs — has been validated in several analysis contexts [12]. For streaming LLM token delivery, Server-Sent Events (SSE) offer a simpler, HTTP/1.1-compatible alternative to WebSockets when only unidirectional server-to-client communication is required [13].

2.8 Literature Synthesis and Research Gap

The literature reviewed above converges on four themes that directly inform FinGuide's architecture:

21

1. **Document/OCR.** Multi-path extraction (text layer → numeric rescue → image OCR) with field-level evaluation is necessary for Hebrew payslips with mixed scripts and vendor-specific layouts [1]–[4], [6].
2. **Hebrew NLP.** Domain-specific label dictionaries and deterministic parsers are appropriate for finite-vocabulary semi-structured documents; general-purpose morphological analysis is secondary [5], [6].
3. **Israeli labor regulation.** Compliance findings must be defined against statutory floors in pension, NII, and study fund law rather than heuristic thresholds [7], [14], [15].
4. **Fintech AI.** Rule-first routing with LLM fallback and multi-agent orchestration balances accuracy and natural-language coverage while mitigating hallucination risk [10]–[13].

Research gap. No prior integrated platform combines (a) automated Hebrew payslip extraction with field-level quality gating, (b) rule-based compliance checking against Israeli statutory minimums across deposit, rate, and continuity dimensions [7], [14], [15], and (c) a hybrid AI advisory layer grounded in the user's extracted payslip history — all within an employee-facing, Hebrew-native web application that does not require banking API access [16], [17]. Employer portals address display but not cross-employer longitudinal analysis; international PFM tools lack Israeli regulatory models; and academic OCR work rarely targets payslip-specific compliance downstream [18]. FinGuide is positioned to fill this gap, with evaluation scope and limitations reported in Chapters 4 and 5.

22

Chapter 3: System Design and Implementation

This chapter describes FinGuide's architecture, data pipeline, and implementation. Section 3.1 presents the high-level design; Sections 3.2 and 3.3 detail preprocessing, extraction, findings, AI, and frontend components; Section 3.4 defines evaluation metrics; and Section 3.5 lists software and hardware specifications.

3.1 System Architecture

FinGuide is organized as a monorepo containing two workspaces: `backend/` (Node.js + Express) and `frontend/` (React 19 + TypeScript). **Why a monorepo:** both workspaces share release cadence and the `finguide-monorepo` workspace dependency, and `concurrently` starts both dev servers from the root. **Alternative considered:** separate repositories with published API contracts. **Trade-off:** the monorepo simplifies local development and cross-cutting refactors (e.g., renaming `analysisData` fields) but couples deployment and requires coordinated versioning of the mapping layer in `documentToPayslip.ts`.

The root `package.json` orchestrates both via `concurrently`, which starts the backend development server on port 5000 and the Vite development server on port 5173 simultaneously. The Vite configuration reverse-proxies `/api` and `/uploads` requests to `127.0.0.1:5000`, eliminating cross-origin request complexity during development.

%%IMG:Figure 1: High-level system architecture::figures/fig01-architecture.png%%

At the top sits the browser client running the React SPA. The client communicates with the Express API server over HTTPS. The API server interacts with four external systems: MongoDB (data persistence), Tesseract and Poppler utilities (OCR processing, run as child processes), the Anthropic Claude API (LLM inference), and optionally an Ollama local server as an LLM fallback. The filesystem stores uploaded PDF files under `backend/uploads/`.

The deployment topology is containerized using Docker Compose for local development via the `dev:docker` command, which provisions a MongoDB container alongside the Node.js server image. The server image is based on a Node.js base image extended with `tesseract-ocr`, `tesseract-ocr-heb`, and `poppler-utils` system packages, which are required for OCR processing and are not present on standard macOS developer machines.

23

3.2 Data Collection and Preprocessing

3.2.1 Document Ingestion

Documents are submitted to the system via the `POST /api/documents/upload` endpoint. Multer middleware validates the uploaded file against two criteria: the MIME type must be `application/pdf` or one of the Microsoft Excel types (`application/vnd.ms-excel`, `application/vnd.openxmlformats-officedocument.spreadsheetml.sheet`), and the file size must not exceed a configurable limit (default 10 MB, governed by the `MAX_UPLOAD_SIZE_MB` environment variable). Files that pass validation are stored with a UUID-generated filename under `backend/uploads/`. A SHA-256 checksum of the file content is computed immediately after storage and recorded on the Document record to enable future deduplication.

%%IMG:Figure 2: Document processing pipeline::figures/fig02-ocr-pipeline.png%%

The upload handler calls `processFinancialDocument`, which creates a MongoDB Document record with `status: 'pending'` and immediately invokes the synchronous extraction pipeline. The pipeline proceeds through up to three extraction paths in order of reliability, falls back to the next path upon failure, and ultimately writes the result back to the Document record as `analysisData` with `status: 'completed'`, `status: 'needs_review'`, `status: 'needs_password'`, or `status: 'failed'`.

3.2.2 Multi-Path Text Extraction

The extraction pipeline attempts three paths in sequence, choosing the highest-quality result:

Path 1 — Direct PDF text extraction (pdf_text). The pipeline first attempts text extraction via Poppler's `pdftotext` utility, which handles Hebrew encoding more reliably than pure JavaScript PDF parsers on many Israeli payroll exports. If `pdftotext` yields fewer than 50 characters, the system falls back to the `pdf-parse` Node.js module. If the combined text contains at least 200 characters and does not exhibit the broken-Hebrew encoding pattern described in Section 2.2, the text is passed directly to `extractPayslipFinancialEN`. An additional quality gate requires that the extracted `analysisData` contains `gross_total > 500` and `net_payable > 500` — without both of these salary fields, the extraction is considered to have failed even if the text was successfully read. The gate is implemented as follows:

24

```
javascript
// payslipOcr.js — Path 1 success requires both core salary fields
const gross = data?.salary?.gross_total;
const net = data?.salary?.net_payable;
if (!(gross > 500 && net > 500)) {
  // fall through to Path 2 or Path 3
}
```

Why synchronous processing: the upload handler runs extraction inside the HTTP request cycle. **Alternative:** `processDocumentAsync` with a job queue (tested but not wired to upload). **Trade-off:** synchronous processing simplifies status handling and avoids polling infrastructure, but multi-page image OCR can add 10–20 seconds of request latency (see Section 5.2).

Path 2 — Numeric rescue. If Path 1 yields text of adequate length but broken Hebrew encoding (as detected by the `isLikelyBrokenHebrew` heuristic, which checks for a ratio of Unicode replacement characters above 1.5% or fewer than 8 Hebrew characters in a 400-character sample), the system still attempts extraction. Hebrew labels cannot be reliably matched, but numeric values — salary amounts, percentages, identification numbers — can often still be extracted from the corrupted text because Arabic numerals survive encoding corruption.

Path 3 — Image-based OCR. When text extraction fails, the system falls back to rendering each PDF page as a 300 DPI PNG image using `pdftoppm`, preprocessing the image with Sharp, and submitting the result to Tesseract.

3.2.3 Image Preprocessing

Before submission to Tesseract, each page image undergoes a preprocessing chain implemented with the Sharp Node.js library:

1. **Auto-rotation.** Sharp applies EXIF-based rotation to correct orientation artifacts from scanning.
2. **Grayscale conversion.** The color image is converted to an 8-bit grayscale representation, eliminating color variation that adds noise without contributing to character recognition.
3. **Contrast normalization.** Sharp's `normalize()` operation stretches the grayscale histogram to the full 0–255 range, improving contrast on faded or low-contrast documents.

25

4. **Thresholding.** A global threshold of 170 binarizes the image: pixels with luminance above 170 become white, those below become black. This produces a clean black-on-white binary image that Tesseract's LSTM network processes most effectively. The threshold value of 170 was determined empirically on a development corpus of payslips.

The preprocessing chain is intentionally simple and deterministic. Adaptive preprocessing (e.g., Otsu's method for automatic threshold selection) was considered but rejected because the uniformly illuminated, printer-generated documents in the target domain consistently benefited from a fixed threshold value.

After preprocessing, the image is submitted to Tesseract with the combined Hebrew and English language model (`heb+eng`), LSTM-based OEM 1, the `preserve_interword_spaces=1` configuration, and three PSM candidates (6, 4, 3) run in sequence. The raw text output from each candidate is scored by `rankExtractionCandidates`, which sorts candidates by resolution score, confidence, and warning count:

```
javascript
// payslipOcrResolver.js — select best OCR candidate
function rankExtractionCandidates(candidates = []) {
  return [...candidates].sort(
    (a, b) =>
                    (b.data?.quality?.resolution_score   ??   0)   -
(a.data?.quality?.resolution_score ?? 0) ||
        (b.data?.quality?.confidence ?? 0) - (a.data?.quality?.confi-
dence ?? 0) ||
                    (a.data?.quality?.warnings?.length   ??   0)   -
(b.data?.quality?.warnings?.length ?? 0),
  );
}
```

The highest-scoring candidate advances to field extraction.

3.2.4 LLM Adjudication for Ambiguous Fields

When multiple extraction candidates are available and their extracted values for a specific field conflict, the system invokes an LLM adjudication step. The `adjudicateField` function submits the conflicting candidates to Claude Haiku 4.5 (`claude-haiku-4-5`) with a structured prompt requesting selection of the most plausible value given the field's domain

26

constraints. The adjudication model returns a chosen value and a confidence score in the range [0, 1]. The final field score is adjusted upward from the base candidate score using the formula:

```
finalScore = max(chosenScore, min(1.0, 0.85 + (confidence - 0.6) × 0.3))
```

This formula ensures that high-confidence adjudications produce field-level scores near 0.85–1.0, providing a calibrated quality signal to downstream components. The use of Claude Haiku for this task (rather than the more capable Sonnet model used for the chat assistant) reflects a deliberate cost-performance trade-off: adjudication requires only short-context field comparison, a task well within Haiku's capabilities at substantially lower cost.

3.2.5 analysisData Canonical Schema

All extraction paths ultimately produce an `analysisData` object conforming to schema version 1.9. This object is stored as a schema-less BSON `Object` in the MongoDB `Document` record. **Why schema-less storage:** the extraction algorithm evolves frequently; new fields are added without relational migrations. **Alternative:** a normalized relational schema with migration scripts. **Trade-off:** MongoDB's `Schema.Types.Mixed` enables rapid iteration and backfill via `npm run reprocess:payslips`, but nested-field queries are not enforced at the database level.

The schema is versioned via the `schema_version` field, allowing the reprocessing script to identify and backfill documents produced by older extraction algorithms.

The top-level structure of `analysisData` v1.9 is:

27

```
{
  schema_version: "1.9",
  period: { month: string },
   salary: { gross_total: number, net_payable: number, components:
[...] },
   deductions: { mandatory: { total, income_tax, national_insurance,
health_insurance } },
  contributions: {
      pension: { employee, employee_amount, employer, base_salary_-
for_pension },
    study_fund: { employee, employer }
  },
  tax: { gross_for_income_tax, tax_credit_points, personal_credit },
  national_insurance: { employee_amount, employer_amount },
   employment: { employer_name, employment_start_date, employment_type
},
  parties: { employer_name, employee_name, employee_id },
  insurances: { ... },
   quality: { score, extraction_path, validation: { issues, warnings }
},
  raw: { rawText, ocr_text },
  summary: { date }
}
```

%%IMG:Figure 4: analysisData canonical schema structure::figures/fig04-analysis-data-schema.png%%

The `quality` sub-object records the extraction path taken, an overall quality score, and a list of validation issues detected during extraction. Documents with quality scores below a threshold are given `status: 'needs_review'` rather than `status: 'completed'`, which triggers a prompt in the UI encouraging the user to manually verify the extracted values.

3.3 Implementation Details

The subsections below trace the data flow from backend request handling through findings detection, AI advisory, and frontend presentation. Each major subsystem includes explicit design rationale where architectural alternatives were considered.

3.3.1 Backend Architecture

%%IMG:Figure 3: Backend layered architecture::figures/fig03-backend-layers.png%%

The backend is structured as five horizontal layers:

Routes (`backend/routes/`). Nineteen Express router modules are mounted in `app.js`: auth, documents, ai, findings, onboarding, profile, insights, recommendations, notifications, Gmail integration, tax-assistant, financial-health, copilot, score-agent, pension, insurance, dashboard, agents, and summary-email. Each module mounts on a URL prefix and delegates request handling to the corresponding controller. Route modules are responsible only for URL-level concerns: HTTP method, URL parameters, middleware application, and controller dispatch.

Controllers (`backend/controllers/`). Controller functions extract validated inputs from the request object, call the appropriate services, and format the HTTP response. Controllers contain no business logic; they translate between HTTP and the internal domain model.

Services (`backend/services/`). Service modules implement business logic. The `financialDocumentService` orchestrates the extraction pipeline. The `savingsForecastService` computes pension projections. The `claudeChatService` manages communication with the Anthropic API.

Utilities (`backend/utils/`). Utility modules implement pure algorithmic logic without external dependencies: the findings detectors (`detectFundWithoutDeposit`, `detectContributionRateGap`, `detectDepositContinuityGap`), the financial health score utilities, and the payslip period normalization functions.

Models (`backend/models/`). Mongoose schema definitions and model constructors. The User model and Document model are described in detail in Section 3.3.4.

Request lifecycle: An incoming HTTP request is processed by the global middleware stack defined in `app.js` — rate limiter (2,000 requests per 15 minutes in development, 100 per 15 minutes in production), CORS whitelisting (CLIENT_URL and localhost:5173), JSON body parsing — before reaching the matching route handler. Protected routes pass through the `protect` middleware, which extracts the Bearer token from the Authorization header, verifies it using `jsonwebtoken.verify`, and attaches the retrieved User document to `req.user`. Error propagation uses Express's `next(error)` pattern; the global `errorHandler` middleware normalizes Mongoose `ValidationError`, `CastError`, duplicate key (error code 11000), JWT verification failures, Multer upload errors, and custom `AppError` subclasses into typed JSON error responses.

3.3.2 Golden-Fixture Regression Testing

The extraction pipeline is regression-tested against a labeled golden corpus stored under `backend/services/__fixtures__/golden/` (seven anonymized payslips spanning Michpal and Malam Plus templates as of July 2026). The `npm run eval:ocr` script up-

29

loads each fixture programmatically, compares extracted fields against ground-truth annotations, and reports per-field accuracy and confusion details. This provides a repeatable baseline for measuring extraction quality as the label dictionary and preprocessing chain evolve. A next-generation extraction architecture (extraction-v2) is under evaluation using additional fixtures under `backend/fixtures/extraction-v2/`; it is not yet wired into the production upload path.

Once a document reaches `status: 'completed'` with a populated `analysisData` object, the findings engine consumes the same canonical schema without re-parsing the PDF. This separation — extract once, analyze many times — keeps compliance logic deterministic and testable independently of OCR variance.

3.3.3 Findings Detection Engine

The findings detection engine is invoked by `GET /api/findings` and produces a structured list of financial anomalies and warnings for the authenticated user. The findings are generated from two layers:

Layer 1 — Metadata findings. Before analyzing payslip content, the engine checks for document-level problems: the user has no uploaded documents at all; there are multiple documents with identical filename and file size (suspected duplicates); one or more documents have remained in `pending` or `processing` status without update for more than 30 days (stale documents); one or more payslips have missing `periodMonth` or `periodYear` metadata; and a payslip has a period date in the future (likely a data entry error).

Layer 2 — Financial findings. Three specialized detectors analyze the content of completed payslips:

Fund deposit detection (`detectFundWithoutDeposit.js`). For each payslip, the detector checks whether a pension or study fund section was identified in the extraction (via the `storedDetection` quality flag, or heuristically from the presence of `base_salary_for_pension`), and whether the sum of employee and employer deposit amounts is zero. A finding is generated if both conditions hold and the document does not exhibit the `missingLine` warning that would indicate the relevant section was simply not present in the payslip. Additionally, the detector compares onboarding-declared fund participation (`user.onboarding.data.hasPension`, `hasStudyFund`) against the latest extracted payslip to detect cases where the user believes they have a fund but no deposits appear.

30

Contribution rate gap detection (`detectContributionRateGap.js`). The detector computes an implied contribution rate as `(amount / base_salary) × 100` for both pension and study fund components. It then compares the implied rate against the stated rate (when present), flagging inconsistencies above a tolerance of 0.35 percentage points. It also compares both the stated and implied rates against the statutory minimums configured in `contributionRateThresholds.js`: pension employee 6.0%, pension employer 6.5%, pension severance 6.0%, study fund employee 2.5%, study fund employer 7.5%. Three finding types are generated: `inconsistency` (stated vs. implied differ beyond tolerance), `belowMinimum` (effective rate below statutory threshold), and `dataIncomplete` (contribution section present but rates are null). The core comparison logic is:

```
javascript
// detectContributionRateGap.js — implied vs stated rate check
const impliedPercent = computeImpliedPercent(amount, base, analysisDa-
ta, thresholds);
const consistencyGap =
  statedPercent != null && impliedPercent != null &&
  Math.abs(statedPercent - impliedPercent) > thresholds.inconsistency-
TolerancePercent;
const effectivePercent = statedPercent ?? impliedPercent;
const belowMinimum =
  effectivePercent != null && minimumPercent != null &&
  effectivePercent < minimumPercent;
```

Deposit continuity gap detection (`detectDepositContinuityGap.js`). The detector constructs a monthly timeline from the sorted payslip history. It flags two types of breaks: *on-payslip gaps*, where a payslip is present for a given period but the deposit column shows zero; and *missing-payslip gaps*, where two deposit-bearing payslips are separated by one or more months for which no payslip was uploaded. An uncertainty counter tracks months in which the payslip is present but the quality score is too low to confidently classify the deposit status.

All findings are assigned a severity level and sorted with `warning` findings listed before informational findings. Each finding includes a `meta` object containing the relevant fund type, the affected period months, the associated document identifiers, and a `findingKind` discriminator, enabling the frontend to render deep-link URLs into the payslip history view with the relevant periods highlighted.

Findings feed both the Hub dashboard and the AI assistant: rule-based intents such as `pension_status` read the same `contributions` fields that the detectors analyze, ensuring consistent answers whether the user views a finding card or asks a conversational question.

%%IMG:Figure 9: Findings detection decision tree::figures/fig09-findings-tree.png%%

The figure illustrates the three-level decision tree for pension deposit detection: (1) Was a pension section identified in extraction? If not, skip. (2) Is the sum of employee and employer deposits zero? If not, skip. (3) Is the missingLine warning absent? If so, emit `PENSION_MISSING_DEPOSIT` finding.

3.3.4 Data Models

User model (`backend/models/User.js`). The user schema stores authentication credentials and onboarding data:

`name`: string, max 100 characters.

`email`: string, unique indexed, lowercase normalized.

`googleId`: string, sparse unique index (null for non-Google users).

`password`: string, `select: false` (never returned in queries by default), minimum 6 characters.

`onboarding`: nested object containing `completed` (boolean), `completedAt` (date), and `data` (a sub-document recording salary type, expected monthly gross, employment start date, pension and study fund participation flags).

`gmailIntegration`: nested object recording Gmail OAuth tokens for the optional email integration feature.

Password hashing is handled by a Mongoose `pre('save')` hook that calls `bcryptjs.genSalt(10)` followed by `bcryptjs.hash` on the plain-text password whenever the password field is modified.

Document model (`backend/models/Document.js`). The document schema represents a single uploaded payslip:

`user`: ObjectId reference to the User, with a compound index on `{user, uploadedAt}` for efficient per-user queries sorted by upload date.

`filename`: string, unique — the UUID filename assigned at upload time.

`checksumSha256`: string — the hex-encoded SHA-256 digest of the file content.

32

`status`: enum `['uploaded', 'pending', 'processing', 'completed', 'needs_review', 'needs_password', 'failed']`.

`analysisData`: `Schema.Types.Mixed` — the schema-less extraction result object.

`processingError`: string — the error message recorded when status is `failed`.

`source`: enum `['manual', 'gmail']` — whether the document was uploaded manually or imported via the Gmail integration.

`emailMetadata`: nested object storing Gmail message and attachment identifiers for deduplicated Gmail imports.

3.3.5 Authentication

%%IMG:Figure 8: Authentication flow::figures/fig08-auth-flow.png%%

The system supports two authentication modes.

Local authentication is implemented using JSON Web Tokens. The `POST /api/auth/register` endpoint accepts a validated name, email, and password (minimum 6 characters, must contain at least one uppercase letter, one lowercase letter, and one digit, per express-validator rules). The user record is created in MongoDB — the pre-save hook hashes the password — and a 7-day JWT is returned. The `POST /api/auth/login` endpoint finds the user by email using `User.findOne().select('+password')` (the `+password` projection overrides the default `select: false`), verifies the password with `bcrypt.compare`, and returns a new JWT on success.

Google OAuth 2.0 is implemented via the `POST /api/auth/google` endpoint. The client submits a Google ID token obtained from the Google Sign-In button. The server verifies the token using `OAuth2Client.verifyIdToken` from the `google-auth-library` package, which validates the token's signature, issuer, expiry, and audience against the configured client ID (`GOOGLE_CLIENT_ID`). The verified payload yields the user's email, name, and Google subject identifier. The server performs an upsert: it searches for a user with a matching `googleId` or email, linking the Google identity to an existing account if one is found, or creating a new account with a random UUID password placeholder if the email is not registered.

All protected API routes pass through the `protect` middleware. JWT verification failure, missing token, or a deleted user account produce standardized 401 responses. Token expiry is configurable via the `JWT_EXPIRE` environment variable; the default is 7 days.

3.3.6 AI Assistant

33

The AI assistant is exposed through the `POST /api/ai/chat` (standard request/response) and `POST /api/ai/chat/stream` (Server-Sent Events streaming) endpoints, and the `GET /api/ai/financial-tips` endpoint.

Intent detection. Before invoking any language model, the `detectIntent` function in `aiController.js` performs keyword-based intent classification over the user's message text. The classifier recognizes approximately 24 distinct intents covering the domain of Israeli personal finance: salary and component questions, pension and study fund queries, tax calculations and credit point questions, what-if salary simulations, month-over-month comparison requests, insurance gap detection, savings forecast requests, onboarding status queries, and general financial health inquiries. **Why rule-first routing:** deterministic handlers return values from the user's payslip data with `source: "rule"`, avoiding LLM hallucination on factual queries [11]. **Alternative:** LLM-first classification for all queries. **Trade-off:** keyword routing is brittle to paraphrase but auditable; the two misclassified queries in the n = 39 evaluation set (Section 4.2.4) illustrate this limitation.

```
javascript
// aiController.js — keyword intent routing (excerpt)
function detectIntent(message) {
  const msg = normalizeMessage(message);
   if (msg.includes('חריג') || msg.includes('anomaly')) return 'anom-
aly_check';
   if (msg.includes('פנסיה') && msg.includes('הפקדה')) return 'pen-
sion_status';
  // ... additional intent patterns
  return 'fallback';
}
```

Rule-based responses. When a recognized intent is matched, `buildRuleBasedAnswer` generates a deterministic, data-grounded response by querying the user's payslip history from the database. For example, the `pension_status` intent retrieves the user's most recent payslip, reads `contributions.pension.employee` and `contributions.pension.employer`, compares against the statutory minimums from `contributionRateThresholds.js`, and returns a Hebrew-language status message with specific shekel amounts and percentage rates taken directly from the document. These responses are labeled `source: "rule"` in the API response.

34

LLM fallback. When a recognized intent is matched but `buildRuleBasedAnswer` cannot produce a data-grounded response (returns `null`), or when intent classification returns `'fallback'`, the `claudeChatService.chat()` function is invoked. The service builds a system prompt that includes the user's complete payslip data (up to 50 documents), active insights and recommendations, pension analysis results, insurance analysis results, and a reference section covering Israeli pension rates, insurance types and approximate costs, 2026 income tax brackets, and current mortgage rate ranges. The user's messages and assistant responses are persisted as `ChatMessage` documents to maintain conversation history, which is included in subsequent API calls to provide context continuity. API responses label the source as `"rule"`, `"claude"`, or `"ollama"` depending on which layer answered.

The primary LLM is Claude Sonnet 4.6 (`claude-sonnet-4-20250514`), selected for its strong performance on Hebrew financial reasoning tasks. The system falls back to an Ollama-hosted local model (default `llama3.1:8b`) when `ANTHROPIC_API_KEY` is not configured, enabling offline development and testing.

Streaming. The `/chat/stream` endpoint uses Node.js's `res.write()` to deliver Server-Sent Events. **Why SSE over WebSockets:** LLM token delivery is unidirectional; SSE requires no handshake and works over standard HTTP [13]. **Trade-off:** SSE does not support client-to-server streaming within the same connection, which is not required for the current chat UX.

The response header sets `Content-Type: text/event-stream` and `Cache-Control: no-cache`. The LLM response is streamed token by token using the `@anthropic-ai/sdk` streaming API, with each token chunk wrapped in an SSE `data:` frame of type `token`. A final `done` event signals stream completion, and error events deliver error metadata to the client for graceful degradation.

3.3.7 Multi-Agent AI Orchestration

%%IMG:Figure 5: Multi-agent AI orchestration pipeline::figures/fig05-multi-agent.png%%

The orchestration layer is exposed through the `POST /api/ai/full-analysis` endpoint. On the Hub page, analysis runs when the user explicitly triggers it (via the Run control in `MasterAgentPanel`); it is not invoked automatically on page load. The pipeline proceeds through four steps:

35

Step 0 — Execution Canvas. `buildExecutionCanvas` loads the user's onboarding data, completed documents, user profile, existing recommendations, and insurance and pension records to construct a domain task inventory. The canvas determines which domain agents need to run (based on `focus`: `'all'`, `'payslip'`, `'insurance'`, or `'pension'`) and records the data availability status for each domain.

Step 0.5 — Government data prefetch and global score (parallel). `prefetchGovMarketData` warms an in-memory cache with Israeli government data from data.gov.il — pension fund market returns, indexed contribution rates, and similar benchmark data — by making HTTP requests to the public API. Concurrently, `buildFinancialHealthScore` computes the user's financial health score for the current year.

The financial health score is a composite of five weighted categories, totaling 100 points:

Document completeness (max 25 points): rewards users who have uploaded payslips for most months of the year.

Salary stability (max 20 points): rewards consistent month-over-month salary, penalizes unexplained drops.

Tax readiness (max 20 points): rewards timely Form 106 availability and absence of tax anomalies.

Pension consistency (max 20 points): rewards regular, above-minimum pension contributions.

Risk insurance (max 15 points): rewards completion of the insurance profile onboarding questionnaire.

%%IMG:Figure 6: Financial health score computation model::figures/fig06-health-score.png%%

Each sub-score is computed by a dedicated function within `financialHealthScoreService.js` using the user's payslip history and profile data.

Step 1 — Domain agents in parallel. Four agent functions run concurrently via `Promise.allSettled`:

`runPayslipAgent`: retrieves payslip summaries, runs salary trend analysis, generates rule-based recommendations, and optionally calls Claude for an LLM explanation.

`runInsuranceAgent`: loads the user's insurance profile and compares declared coverage types against benchmark coverage expectations for the user's income bracket.

36

`runPensionAgent`: fetches pension data from the pension database model and PensiaNet comparison data, evaluates contribution adequacy, and generates recommendations.

`runFinancialProfileAgent`: analyzes the user's overall financial profile across all domains.

Each agent follows the same internal pipeline: fetch data via tool functions → run deterministic analysis → generate rule-based recommendations → optionally call Claude for an LLM-generated narrative explanation. Agents return structured DTOs; they never pass raw database documents to the LLM.

Step 2 — Recommendation merging and action items. `mergeRecommendations` deduplicates and ranks recommendations from all four agents by priority and domain overlap. `buildActionItems` selects the highest-priority subset and formats them as concrete actions with urgency indicators — bridging automated analysis to user-visible next steps on the Hub.

Step 3 — Orchestrator summary. The orchestrator assembles a safe context object (containing no raw credentials or full document text) from the canvas, government data, global score, action items, and agent results. This context forms the system prompt for a final Claude call that generates a unified Hebrew-language narrative summary of the user's financial health and most important next actions. If Claude is unavailable or `skipLLM` is set, the `generateHebSummary` fallback in `explanationAgent.js` produces a rule-based summary from the agent results.

The entire analysis run is logged to an `AgentRunLog` document recording the run identifier, start time, duration, agent results, and final summary, enabling audit and debugging.

3.3.8 Savings Forecast

The savings forecast module (`POST /api/findings/savings-forecast`) implements a linear projection model for pension accumulation:

```
projectedBalance = currentBalance + monthlyContribution × monthsToRetirement
```

The model is intentionally simple: it does not account for investment returns on the accumulated balance, inflation, or changes in contribution rate over the forecast horizon. This design choice prioritizes transparency and auditability over false precision; users are informed of the model's assumptions in the UI.

37

`savingsForecastService` resolves the monthly contribution input from one of two sources in order of priority: the `pension.employee` plus `pension.employer` fields from the most recent completed payslip with non-zero pension contributions; or an explicit `currentMonthlyContribution` parameter provided by the user in the request body. If neither source is available, the endpoint returns HTTP 400. The service generates two scenarios — current trajectory (unmodified monthly contribution) and an adjusted scenario (with a user-specified increased contribution) — and returns a yearly timeline of projected balances for each scenario, enabling a comparison chart in the frontend.

3.3.9 Frontend Architecture

%%IMG:Figure 7: Frontend route tree::figures/fig07-route-tree.png%%

The React 19 application is organized around a route tree declared in `App.tsx`. Public routes (accessible without authentication) include the landing page, login, registration, and password reset. All other routes are wrapped in `RequireAuth`, a route guard component that reads the authentication state from `AuthProvider` and redirects unauthenticated users to the login screen.

The primary application areas accessible after authentication are:

Hub (`/hub`): The main dashboard; the user triggers multi-agent analysis via the Run control and the page displays the health score, key findings, recent payslips, and action items.

Documents (`/documents`): The document upload area with drag-and-drop support, OCR status indicators, and document management actions.

Payslip History (`/documents/history`): A chronological view of all uploaded payslips with salary trend visualizations implemented primarily as custom inline SVG components with CSS animation, providing pixel-level control over the Hub trend chart.

Pension (`/pension`): A dedicated view combining extracted payslip pension data with PensiaNet market comparison.

Insurance (`/insurance`): The insurance coverage assessment page.

Financial Planning (`/planning`): The savings forecast calculator.

AI Assistant (`/assistant`): The conversational chat interface with streaming response support.

Tax Assistant (`/tax`): A Form 106 and tax-year summary view.

Financial Health (`/financial-health`): The detailed health score breakdown.

38

The application entry (`main.tsx`) composes four nested providers: `BrowserRouter` → `AuthProvider` → `AiChatProvider` → `App`. `AuthProvider` calls `GET /api/auth/me` on mount to restore an authenticated session from the JWT stored in `localStorage`. `AiChatProvider` manages the global floating AI assistant state — conversation history, streaming SSE connection, and voice input — so that the chat panel remains accessible on every authenticated page without re-mounting. The API client (`client.ts`) reads the JWT on every request and attaches it as a `Bearer` token in the `Authorization` header. All UI strings are in Hebrew, and the document root receives `direction: rtl` at the page container level to ensure correct bidirectional text rendering.

The single mapping layer between the backend `analysisData` structure and the frontend UI types is `documentToPayslip.ts`. This module exports two primary mappers: `documentToPayslipHistoryItem` (for the compact list view) and `documentToPayslipDetail` (for the detailed payslip view), both of which translate the snake_case backend field names to camelCase TypeScript types. Changes to the backend schema are propagated to the frontend exclusively by editing this file, preventing drift between the two representations. The frontend never reads raw OCR text or internal `quality.debug` fields — those are stripped by `documentSerializer.js` before API responses leave the backend.

3.3.10 Extended Domain Modules

Beyond the core payslip pipeline, FinGuide implements additional REST domains mounted in `app.js`:

Insights and recommendations (`/api/insights`, `/api/recommendations`). The insights module runs payslip trend analysis via `POST /api/insights/run` and persists structured insight records with severity and dismissal state. The recommendations module executes rule-based insurance and pension recommenders through `POST /api/recommendations/run`, surfacing actionable items the Hub and assistant can reference.

Copilot (`/api/copilot`). Provides a consolidated financial snapshot via `GET /api/copilot/analysis` (profile, latest payslip, budget, investments, health score, goals) and supports goal management and monthly markdown reports generated by Claude with a rule-based fallback.

Gmail integration (`/api/integrations/gmail`). Optional OAuth connection allowing users to import payslip PDF attachments from Gmail (`POST /connect`, `POST /sync`), reducing manual upload friction while keeping credentials scoped to the user's account.

39

Dashboard aggregation (`/api/dashboard/summary`). Combines documents, profile, policies, and recommendations in a single `Promise.all` response for overview screens.

These modules share the same authentication, error-handling, and MongoDB persistence patterns described in Section 3.3.1; their detailed route tables appear in Appendix A.

3.4 Evaluation Metrics

The system is evaluated along three dimensions, using the same thresholds reported in Chapter 4:

OCR accuracy. Field-level extraction rate on the golden fixture corpus (n = 7): a numeric field counts as correct when its value is within **0.5%** of the annotated ground truth (`npm run eval:ocr`). String fields require exact match.

Findings precision and recall. On the annotated scenario corpus (n = 7), precision is the fraction of generated findings that correspond to genuine anomalies; recall is the fraction of annotated anomalies detected (`npm run eval:findings`).

AI assistant routing. Intent classification accuracy on the Hebrew query set (n = 39): a query is correct when `detectIntent()` returns the expected intent label (`npm run eval:ai-routing`). Rule-based responses are additionally checked for factual accuracy against payslip fixtures.

3.5 Software and Hardware Specifications

Backend software (from `backend/package.json`):

40

Component	Version
Node.js runtime	20.x (Docker base image)
Express	^4.18.2
Mongoose	^8.0.3
Jest	^30.2.0
pdf-parse	^1.1.4
Sharp	^0.34.5
@anthropic-ai/sdk	^0.97.1
bcryptjs, jsonwebtoken, multer	^2.4.3 / ^9.0.2 / ^1.4.5-lts.1

Frontend software (from `frontend/package.json`):

Component	Version
React	^19.2.0
TypeScript	~5.9.3
Vite	^7.2.4
react-router-dom	^7.13.0
Jest + ts-jest	^30.2.0 / ^29.4.6

System binaries (OCR pipeline): Tesseract OCR with Hebrew language pack (`tesseract-ocr-heb`), Poppler utilities (`pdftoppm`, `pdftotext`). These are bundled in the backend Docker image (`dev:docker`); on macOS development hosts they must be installed separately or accessed via Docker.

Hardware and deployment environment: Development was carried out on macOS and Linux workstations. Docker Compose provisions MongoDB and the backend with OCR dependencies. Uploaded PDFs are stored on the local filesystem under `backend/uploads/` (not object storage). Production-oriented constraints include synchronous upload processing latency, dev rate limits of 2,000 requests per 15 minutes (100 in production), and optional Ollama (`llama3.1:8b`) as an LLM fallback when `ANTHROPIC_API_KEY` is unset.

41

3.6 Security and Data Privacy

Payslips (תלושי שכר) contain sensitive personal and financial data — national ID numbers, salary amounts, employer names, and contribution details. FinGuide treats this data as confidential employee information subject to access control, transport security assumptions, and informational-use disclaimers (see §1.4).

Authentication. Local accounts use bcrypt-hashed passwords (`bcryptjs`, salt rounds 10) with the password field stored as `select: false` on the User model so it is never returned in API responses. JSON Web Tokens are issued on register/login with `JWT_SECRET` (minimum 10 characters, validated at server boot in `server.js`). Google OAuth 2.0 verifies ID tokens via `OAuth2Client.verifyIdToken`, checking signature, issuer, expiry, and audience against `GOOGLE_CLIENT_ID`. All non-auth routes pass through the `protect` middleware, which attaches `req.user` or returns HTTP 401.

Authorization and data isolation. Every Document record is indexed by `user`; controllers scope queries to `req.user._id`. Download endpoints resolve filesystem paths and reject traversal: `resolvedPath.startsWith(uploadsDir)` is required before streaming a PDF (`documentController.js`). Users cannot access another user's payslips or `analysisData`.

AI data grounding. The assistant's `buildUserContext(userId)` loads payslips, profile, insights, and recommendations exclusively from the database for the authenticated user. Client-supplied `userData` in chat requests is not used as a trusted financial source — preventing clients from injecting fabricated payslip values into rule-based answers.

Storage and transport. PDFs are stored locally under `backend/uploads/{uuid}.pdf` with SHA-256 checksums for deduplication. The API is intended to run behind HTTPS in production; development uses localhost with Vite proxying `/api`. Gmail OAuth tokens, when enabled, may be encrypted using `GOOGLE_TOKEN_ENCRYPTION_KEY` or `JWT_SECRET` via `tokenCrypto.js`.

Rate limiting and boot validation. `express-rate-limit` applies 2,000 requests per 15 minutes in development and 100 in production (`app.js`). `server.js` refuses to start without `JWT_SECRET` (≥10 chars) and `MONGODB_URI`, reducing misconfiguration risk.

42

Privacy posture. FinGuide does not sell or share payslip data with third parties beyond configured LLM providers (Anthropic/Ollama) when the user invokes the assistant; those calls include payslip context in the system prompt. Outputs are informational only and do not constitute certified financial, tax, or legal advice (§1.4, §5.2).

43

Chapter 4: Results and Analysis

This chapter reports evaluation results against the objectives stated in Section 1.3. Section 4.1 describes the experimental setup and reproducibility harnesses; Sections 4.2 and 4.3 present and interpret measured outcomes; Section 4.4 compares FinGuide with existing alternatives; and Section 4.5 discusses whether the project objectives were met.

4.1 Experimental Setup

Evaluation was conducted in a development environment on macOS with Node.js 20.x, MongoDB (local or Atlas via `MONGODB_URI`), and Poppler/Tesseract available either natively or through `dev:docker`. No production load test was performed. All reproducible metrics are produced by `npm run eval:ocr`, `npm run eval:findings`, `npm run eval:ai-routing`, and `npm run bench:upload-latency`, supplemented by the Jest unit and integration test suite described below. Rate limiting during development is set to 2,000 requests per 15 minutes (100 in production), which did not constrain the evaluation runs.

4.1.1 Automated Test Suite

The backend test suite comprises **109** `*.test.js` files (excluding `node_modules`), executed against an in-process MongoDB instance provided by `mongodb-memory-server` (v11). Jest 30 runs with `--runInBand` to prevent race conditions from concurrent database writes. Tests are organized under:

`backend/__tests__/` — unit tests for parsers, OCR resolver, and service modules

`backend/tests/unit/` — additional unit tests for utilities and controllers

`backend/tests/integration/` — end-to-end API tests via Supertest (auth, documents, findings, pension, recommendations)

The frontend test suite comprises **3** test files under `frontend/src/`, running in jsdom with ts-jest and `@testing-library/react` 16. Root `npm test` also runs `vite build`, enforcing TypeScript compile checks.

Not covered: browser-based end-to-end (Playwright/Cypress) tests, production load tests, and formal penetration testing. OCR quality on unseen vendors relies on golden fixtures plus manual exploratory review.

The key test files and what each covers are:

44

File	Coverage focus
`auth.routes.test.js`	Registration, login, Google OAuth, password reset flows
`documents.uploadMetadata.test.js`	File upload, metadata assignment, status transitions
`payslipOcrParser.test.js`	Amount parsing, period parsing, Hebrew label recognition
`payslipOcrResolver.test.js`	Candidate ranking, PSM selection, gross/net disambiguation
`documentProcessingService.test.js`	Pipeline state machine (pending → completed / failed)
`findings.savingsForecast.test.js`	Linear forecast model, edge cases (zero balance, same-year retirement)
`payslipHistoryAggregationService.test.js`	Annual aggregation, monthly bucketing, missing-month detection
`detectSalaryAnomalies.test.js`	Month-over-month anomaly detection, outlier flagging
`documentToPayslip.test.ts`	`analysisData` → `PayslipHistoryItem` mapping correctness

4.1.2 OCR Evaluation Framework

The golden fixture corpus comprises **n = 7** anonymized Israeli payslip PDFs from Michpal and Malam Plus payroll systems (July 2026), stored under `backend/services/__fixtures__/golden/`. Each fixture has a companion `expected.json` annotation file with ground-truth values for primary financial fields.

Annotation protocol. Two team members independently annotated each fixture by reading the PDF and recording `gross_total`, `net_payable`, deduction line items, and identifiers. Disagreements were resolved by a third review against the source PDF. Personal identifiers in fixtures were anonymized; only payroll-vendor structure and numeric values needed for field matching were retained.

45

Each document is processed via `extractPayslipFile`. A numeric field counts as correct when within **0.5%** of ground truth; string fields require exact match. Reproduction: `cd backend && npm run eval:ocr`.

Password-protected PDFs are pre-unlocked before evaluation. The corpus consists entirely of digital PDFs with intact text layers; scanned payslips are not yet represented (Section 5.2).

4.1.3 Findings Detection Evaluation Framework

The findings evaluation corpus comprises **n = 7** synthetic scenarios in `backend/scripts/fixtures/findings-eval/scenarios.json`, annotated with expected finding kinds (`deposit`, `rate`, `continuity`) plus negative controls. Scenarios include: study fund zero deposit, compliant pension control, stated-vs-implied rate gap, missing-month continuity gap, pension zero deposit, employer rate below statutory minimum, and on-payslip zero-deposit month.

Ground truth was defined by applying the statutory rules in Section 2.4 to each `analysisData` payload. Reproduction: `cd backend && npm run eval:findings`.

4.1.4 AI Assistant Evaluation Framework

The AI routing evaluation set comprises **n = 39** Hebrew queries in `backend/scripts/fixtures/ai-routing-eval/queries.json`, spanning salary, pension (employee/employer/total), tax, National Insurance, vacation/sick days, documents, notifications, recommendations, insurance profile, what-if, anomaly, and open-ended advisory intents. Each query is classified by `detectIntent()` and compared against the annotated expected intent. Reproduction: `cd backend && npm run eval:ai-routing`.

4.1.5 Regression and Continuous Verification

%%IMG:Figure 11: Verification and regression test flow::figures/fig11-verification.png%%

Regression is enforced at three layers: (1) Jest unit and integration tests on every `npm test`; (2) golden-fixture OCR evaluation via `npm run eval:ocr`; (3) annotated scenario sets for findings and AI routing. The `analysisData` schema version (`1.9`) is treated as a stable contract — changes to extraction must pass golden fixtures before merge.

46

Latency benchmarking. Synchronous Path-1 extraction latency was measured on the golden corpus using `npm run bench:upload-latency` (see Table 4). This characterizes development-machine performance only; production latency under concurrent uploads was not evaluated.

4.2 Presentation of Results

4.2.1 Document Processing Pipeline Outcomes

The pipeline assigns one of four terminal statuses to every uploaded document:

`completed`: All critical fields (`gross_total`, `net_payable`, mandatory deductions, `period.month`) were extracted with confidence above the quality threshold. The document is fully usable for analysis and findings detection.

`needs_review`: Processing succeeded but at least one critical field is missing or below the confidence threshold. The document is partially usable; the frontend presents the extracted fields alongside the original PDF and prompts the user to verify or fill in missing values via `PATCH /api/documents/:id/fields`.

`needs_password`: The PDF is encrypted. The document enters this state before any extraction is attempted; the user provides the password via the unlock flow and the pipeline re-runs.

`failed`: An unrecoverable error occurred (corrupt PDF, missing Tesseract binary, unhandled exception). The `processingError` field records the failure reason.

The `needs_review` status is the key graceful-degradation mechanism: it ensures that even partial extractions are preserved and can be corrected through manual field entry, rather than silently discarding the document.

%%IMG:Figure 10: OCR accuracy results by extraction path::figures/fig10-ocr-results.png%%

On the seven-fixture golden corpus evaluated in July 2026, all documents (100%) were resolved via Path 1 direct text extraction (`pdf_text` via `pdftotext` / `pdf-parse`). No documents required Path 2 numeric rescue or Path 3 image-based OCR. Average extraction confidence was 0.653 and average resolution score was 12.36 across fixtures.

Table 4: Document processing latency — Path 1 extraction (`npm run bench:upload-latency`, n = 7, July 2026)

Fixture	Latency (ms)
michpal-202209	11
michpal-202210	11
michpal-202211	11
michpal-202212	12
malam-plus-202512	90
malam-plus-202601	31
malam-plus-202602	31
Median	**12**
Mean	**28**
Min / Max	**11 / 90**

Malam Plus fixtures exhibit higher latency due to larger PDF size and more complex layout parsing, but all remain under one second on the development machine. Multi-page image OCR (Path 3) was not exercised on this corpus; Section 5.2 notes the synchronous processing ceiling for that path.

4.2.2 Field-Level Extraction Accuracy

Table 1: Field extraction accuracy on golden fixture corpus (n = 7, July 2026)

48

Field	Correct	Mismatch	Missing	Accuracy
`period_month`	7	0	0	100.0%
`gross_total`	7	0	0	100.0%
`net_payable`	7	0	0	100.0%
`employee_id`	7	0	0	100.0%
`tax_credit_points`	7	0	0	100.0%
`base_salary`	3	0	4	100.0% (of applicable)
`mandatory_total`	4	3	0	57.1%
`national_insurance`	4	3	0	57.1%
`health_insurance`	4	3	0	57.1%
`income_tax`	—	—	7	n/a (not annotated)
`personal_credit`	—	—	7	n/a (not annotated)

Core salary fields (`gross_total`, `net_payable`, `period_month`) achieved perfect accuracy on this corpus. Deduction line items (`mandatory_total`, `national_insurance`, `health_insurance`) showed systematic mismatches on three Malam Plus fixtures, where extracted values conflated adjacent deduction rows — indicating that label disambiguation on multi-row deduction tables remains the primary extraction weakness. Expanding the corpus beyond seven fixtures and additional payroll vendors is necessary before drawing generalization claims.

4.2.3 Findings Detection

Table 2: Findings engine precision and recall (annotated scenario corpus, n = 7, July 2026)

49

Metric	Value
Scenarios evaluated	7
Finding kinds tested	deposit, rate, continuity
True positives	6
False positives	0
False negatives	0
Precision	100.0%
Recall	100.0%

Scenarios cover study fund zero deposit, compliant pension control, rate inconsistency, continuity gap, pension zero deposit, employer below-minimum rate, and on-payslip zero-deposit month. This corpus validates detector logic on synthetic `analysisData` payloads; expanding to manually reviewed real payslips is recommended before broad generalization claims.

The findings engine applies a conservative definition of compliance gaps: any calendar month between two deposit-bearing payslips for which no payslip exists in the system is flagged as a potential continuity gap. This design choice favors recall over precision — a false positive (spurious gap alert) requires only user confirmation to dismiss, while a false negative (missed genuine gap) may allow employer non-compliance to go undetected.

4.2.4 AI Assistant Intent Classification

Table 3: AI assistant intent routing accuracy (Hebrew query set, n = 39, July 2026)

Metric	Value
Queries evaluated	39
Intent classification correct	37 (94.9%)
Misclassified	2
Routed to rule-based layer	33
Routed to LLM fallback	6

50

The expanded evaluation set adds pension employee/employer intents, vacation and sick days, documents summary, notifications, recommended actions, and additional what-if phrasing. Two misclassifications remain:

1. *"האם יש לי ביטוח חיים מספיק בגיל 45?"* — classified as `profile_insurance` (expected `fallback`) because the `האם יש לי` pattern matches before advisory context is considered.
2. *"מה מצב הביטוחים שלי?"* — classified as `fallback` (expected `profile_insurance`) because the query lacks the `יש לי ביטוח` trigger phrase.

These cases illustrate keyword-routing brittleness for compound insurance questions (Section 5.3).

4.2.5 Error Analysis — Malam Plus Deduction Fields

The 57.1% accuracy on `mandatory_total`, `national_insurance`, and `health_insurance` is not a character-level OCR failure — Path 1 text extraction succeeds on all Malam Plus fixtures. The errors arise in **label disambiguation** within `payslipOcrResolver.js`: multi-row deduction tables place National Insurance, health levy, and income tax on adjacent lines with similar Hebrew labels. The resolver's `resolveMandatoryTotalCandidate` and related numeric candidate collectors sometimes attribute the summed mandatory block to the wrong row when labels repeat or when Malam Plus uses abbreviated column headers (see Figure 10 for per-field accuracy).

Mitigation path: expand the label dictionary for Malam Plus-specific deduction headers, add row-level spatial heuristics, or promote extraction-v2 layout-aware parsing once it exceeds the rule baseline on an expanded corpus.

4.3 Data Analysis and Interpretation

The results are interpreted against the six objectives defined in Section 1.3.

51

Objective 1 — Automated payslip ingestion and OCR (partially met). Core salary fields (`gross_total`, `net_payable`, `period_month`) achieved 100% accuracy on the n = 7 golden corpus, validating Path 1 text extraction for Michpal and Malam Plus digital PDFs. Deduction line items (`mandatory_total`, `national_insurance`, `health_insurance`) reached only 57.1% due to systematic label-disambiguation failures on Malam Plus multi-row deduction tables — not OCR character recognition failures, but parser logic that conflates adjacent rows. Objective 1 is met for primary salary extraction; deduction-level accuracy requires corpus expansion and label-map refinement.

Objective 2 — Financial findings detection (met on test corpus). The n = 7 scenario set achieved 100% precision and recall on deposit, rate, and continuity kinds.

Objective 3 — Personalized AI assistant (largely met). Intent routing reached 94.9% (37/39) on the expanded Hebrew query set. Two misclassifications involve compound insurance adequacy questions (Section 4.2.4).

Objectives 4–6 — Multi-agent analysis, savings forecast, Hebrew UI (implemented; not quantitatively evaluated). The multi-agent orchestration, linear savings forecast, and RTL React interface are implemented and covered by integration tests, but no automated end-to-end user-study metrics were collected. Their contribution is architectural and functional rather than numerically benchmarked in this project book.

Pipeline design validation. The three-path OCR design is structurally sound: Path 1 resolved 100% of the current corpus. Path 2 and Path 3 remain untested on this corpus because no scanned or encoding-corrupted fixtures were included. The multi-candidate PSM strategy and `rankExtractionCandidates` scoring provide a principled basis for Path 3 selection when image OCR is required.

Findings and AI coupling. The findings engine and AI assistant read the same `analysisData` contract, ensuring consistent answers whether the user views a finding card or asks a conversational question. Hybrid rule-first routing limits hallucination risk on factual queries [11].

4.4 Comparison with Existing Approaches

The following comparison follows a structured format: each alternative's strength, FinGuide's advantage, and FinGuide's limitation relative to that alternative.

Manual payslip review

52

Strength: Requires no software; the employee can spot obvious gross/net errors immediately.

FinGuide advantage: Automates implied-rate computation, deposit timeline construction, and statutory minimum checks that employees rarely perform manually (Objective 2).

FinGuide limitation: Depends on extraction accuracy; manual review may catch layout-specific errors the parser misses (e.g., Malam Plus deduction rows).

Employer payslip portals (Hilan, iCount)

Strength: Authoritative, structured display of each payslip as issued by the payroll system; no upload required when accessed through the employer.

FinGuide advantage: Cross-period longitudinal analysis, compliance findings, and AI advisory across employers in a single Hebrew-native view (Objectives 2, 3, 6).

FinGuide limitation: Requires PDF upload; does not replace the employer portal as the source of record.

Generic personal financial management applications

Strength: Bank transaction aggregation provides spending visibility and budgeting across accounts.

FinGuide advantage: Domain-specific payslip parsing and Israeli regulatory compliance checking unavailable in transaction-based PFM tools (Objectives 1, 2).

FinGuide limitation: No banking API integration; cannot reconcile payslip net salary against bank deposits automatically.

Tax advisors and accountants

Strength: Professional judgment, certification, and holistic tax planning including Form 106 and annual filings.

FinGuide advantage: Continuous monthly monitoring at no marginal cost per review; immediate findings on each uploaded payslip (Objectives 2, 5).

FinGuide limitation: Informational only — not certified financial or tax advice; linear savings forecast omits investment returns and inflation.

Table 5: Capability comparison matrix (qualitative)

53

Capability	Manual	Hilan / iCount	Bank PFM	Accountant	FinGuide
Payslip parse	Partial	Yes	No	Yes	Yes
Compliance checks (deposit/rate/continuity)	Partial	No	No	Yes	Yes
Cross-employer longitudinal view	No	No	Partial	Yes	Yes
Hebrew AI advisory	No	No	No	Partial	Yes
No banking API required	Yes	Yes	No	Yes	Yes
Certified professional advice	No	No	No	Yes	No

%%IMG:Figure 12: Capability comparison summary::figures/fig12-comparison.png%%

The matrix summarizes §4.4 in tabular form. FinGuide's measured advantages (Table 1 salary fields, Table 2 findings on n = 7 scenarios) support the compliance and parsing rows; the accountant column remains stronger on certified advice, which FinGuide explicitly disclaims (§1.4).

4.5 Discussion of Findings

Relative to the primary project aim — enabling Israeli employees to verify, analyze, and act on payslip data without specialist knowledge — the evaluation supports three conclusions.

First, **automated Hebrew payslip extraction is feasible** for digital PDFs from major payroll vendors, with perfect accuracy on core salary fields in the evaluated corpus. The remaining extraction weakness is localized to multi-row deduction tables, a parser refinement problem rather than a fundamental OCR limitation.

Second, **rule-based compliance checking is reliable** on annotated scenarios when extraction output is complete. The 100% precision and recall on n = 7 synthetic cases demonstrates that the three detectors (deposit, rate, continuity) implement the regulatory models in Section 2.4 correctly [7], [14], [15]. Confidence in real-world deployment requires expanding the findings corpus beyond synthetic payloads.

54

Third, **hybrid AI routing is effective but not complete**. The 94.9% intent accuracy on n = 39 queries validates the rule-first design for structured Hebrew financial queries, while two remaining misclassifications on compound insurance questions indicate that keyword routing needs either priority rules for `fallback` or embedding-based classification (Section 5.3).

The project objectives are **substantially met** for extraction (core fields), findings (on test corpus), and AI routing (94.9%), with acknowledged scope limits on corpus size, deduction parsing, and absence of quantitative UX evaluation for the multi-agent and planning features. These outcomes address the research gap stated in §2.8: an integrated Hebrew payslip compliance and advisory platform for employees, without banking API dependency [16], [17].

55

Chapter 5: Conclusion and Future Work

5.1 Summary of Contributions

This project addressed the problem stated in Section 1.2 — Israeli employees' inability to independently verify payslip compliance and act on financial data — through six objectives. The contributions map to those objectives as follows:

1. **Multi-path Hebrew PDF payslip extraction pipeline** (Objective 1). Combines `pdftotext` / `pdf-parse` direct extraction, numeric rescue, and Tesseract image OCR with PSM multi-candidate ranking, LLM adjudication for conflicting fields, and the `analysisData` v1.9 canonical schema. Golden-fixture evaluation reported 100% accuracy on core salary fields (n = 7).

1. **Rule-based financial findings engine** (Objective 2). Three detectors — missing fund deposits, contribution rate gaps, statutory minimum violations, and deposit continuity — achieved 100% precision and recall on the n = 7 annotated scenario corpus.

1. **Hybrid AI assistant** (Objective 3). Keyword intent routing (24 categories) with rule-based data-grounded responses and Claude/Ollama fallback; 94.9% intent classification accuracy on n = 39 Hebrew queries; SSE streaming for conversational delivery.

1. **Multi-agent financial analysis orchestration** (Objective 4). Four parallel domain agents (payslip, insurance, pension, financial profile) with recommendation merging, a 100-point composite health score, and orchestrator narrative generation via `POST /api/ai/full-analysis`.

1. **Longitudinal financial planning** (Objective 5). Linear savings forecast module with scenario comparison, grounded in extracted pension contribution rates from the most recent completed payslip.

1. **Hebrew-native RTL web application** (Objective 6). React 19 and TypeScript SPA with Hub dashboard, payslip history, findings deep-links, AI assistant, and onboarding profile capture — all UI copy in Hebrew with `direction: rtl`.

56

5.2 Limitations

Evaluation corpus size. OCR accuracy (n = 7), findings precision/recall (n = 7), and AI routing (n = 39) were measured on small, manually constructed corpora. Results validate the implementation on representative cases but do not support broad generalization across all Israeli payroll vendors and employment configurations.

OCR completeness. Deduction line items showed 57.1% accuracy on Malam Plus fixtures due to multi-row label disambiguation. Non-standard employer layouts and scanned payslips may yield `needs_review` status requiring manual field entry.

Extraction quality gate. The requirement that both `gross_total > 500` and `net_payable > 500` be present is a pragmatic heuristic that may reject legitimate edge cases (e.g., zero-gross unpaid-leave months).

Synchronous processing. Upload extraction runs inside the HTTP request cycle. Multi-page image OCR may add 10–20 seconds of latency; `processDocumentAsync` exists but is not wired to the upload flow.

Schema-less `analysisData`. MongoDB `Schema.Types.Mixed` storage enables rapid schema iteration but sacrifices database-level query enforcement on nested extraction fields.

LLM knowledge cutoff. The assistant may cite outdated regulatory figures unless overridden by system-prompt constants; users must verify specific statutory amounts.

Linear savings forecast. The projection model omits investment returns and inflation, potentially underestimating terminal pension balances despite UI disclaimers.

Scalability and load testing. The system has not been load-tested under concurrent upload scenarios. Development rate limits (2,000/15 min) do not reflect production constraints.

Israeli regulatory specificity. Findings detectors are coupled to 2026 Israeli statutory floors; adaptation to other jurisdictions would require regulatory model replacement.

No banking API integration. Payslip PDF upload is the sole automated data source; bank and pension portal connections remain future work.

5.3 Future Work

Future work is prioritized by expected impact on the objectives in Section 1.3:

57

High impact — extraction and scale

1. *Asynchronous processing.* Integrate `processDocumentAsync` with a job queue (Bull/BullMQ) to decouple upload latency from OCR duration and enable retry on transient failures.
2. *Expanded golden corpus.* Grow the OCR and findings evaluation sets across Hilan, iCount, Priority, and scanned payslips to strengthen generalization claims.
3. *Extraction-v2 promotion.* Promote the offline-evaluated next-generation extractor to production once it exceeds the rule-based baseline on an expanded corpus.

Medium impact — advisory depth

1. *PensiaNet integration.* Surface pension fund fee and return comparisons from government registry data in the findings engine.
2. *Embedding-based intent classification.* Replace or augment keyword routing to handle compound advisory questions (e.g., the misclassified life-insurance adequacy query in Section 4.2.4).
3. *Banking and pension portal APIs.* Connect to Israeli Open Banking and fund management portals for automated data import.

Lower priority — scope expansion

1. *Annual tax return assistance.* Extend Form 106 parsing and tax-year summary beyond basic metadata.
2. *Compounding savings model.* Add optional investment-return scenarios to the forecast module with explicit assumption disclosure.
3. *Machine learning for field extraction.* Fine-tuned Hebrew table extraction or layout models (e.g., LayoutLM variants) to improve non-standard payslip layouts.
4. *Multi-language support.* Arabic-language OCR and UI to expand coverage beyond Hebrew-primary users.

58

References

[1] Smith, R. (2007). An overview of the Tesseract OCR engine. *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*, vol. 2, pp. 629–633. IEEE. https://doi.org/10.1109/ICDAR.2007.4376991

[2] Tesseract OCR Contributors. (2018). Tesseract 4.x: LSTM-based text recognition engine. Open-source project, Google. Available: https://github.com/tesseract-ocr/tesseract

[3] Nagy, G. (2000). Twenty years of document image analysis in PAMI. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(1), 38–62.

[4] Trier, Ø. D., & Jain, A. K. (1995). Goal-directed evaluation of binarization methods. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 17(12), 1191–1201.

[5] Goldberg, Y. (2017). *Neural Network Methods for Natural Language Processing*. Synthesis Lectures on Human Language Technologies. Morgan & Claypool Publishers. (Discusses morphologically rich languages including Hebrew in the context of NLP model design.)

[6] Xu, Y., Li, M., Cui, L., Huang, S., Wei, F., & Zhou, M. (2020). LayoutLM: Pre-training of text and layout for document image understanding. *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 1192–1200.

[7] Israel Ministry of Finance. (2008). *Pension Obligation Law (חוק הפנסיה החובה)*. State of Israel. Published in Reshumot (Official Gazette) 2156, 2008.

[8] Lusardi, A., & Mitchell, O. S. (2014). The economic importance of financial literacy: Theory and evidence. *Journal of Economic Literature*, 52(1), 5–44.

[9] Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation, University of California, Irvine). Chapter 5 defines the REST architectural style.

[10] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... & Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.

[11] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., ... & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 1–38.

[12] Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, C., ... & Zhang, D. (2023). The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*. (Survey of LLM agent architectures including orchestration and tool-use patterns relevant to multi-domain financial analysis.)

[13] World Wide Web Consortium (W3C). (2015). *Server-Sent Events specification*. W3C Recommendation. https://www.w3.org/TR/eventsource/

[14] Israel Tax Authority. (2026). *Income tax rates, brackets, and credit points for the 2026 tax year*. Israel Ministry of Finance. https://www.gov.il/he/departments/topics/income_tax

[15] National Insurance Institute of Israel. (2026). *National Insurance and health levy contribution rates*. State of Israel. https://www.btl.gov.il/

[16] OECD. (2020). *OECD/INFE 2020 International Survey of Adult Financial Literacy*. OECD Publishing. https://www.oecd.org/financial/education/

[17] Bank of Israel. (2024). *Open banking and payment services — regulatory framework*. Bank of Israel. https://www.boi.org.il/

[18] Wintner, S. (2000). Hebrew computational linguistics: Challenges and directions. *Natural Language Engineering*, 6(1), 1–25.

60

Appendix A: API Endpoint Reference

The following table lists all API endpoints exposed by the FinGuide backend. All endpoints except those under `/api/auth` and `GET /api/health` require a valid JWT Bearer token in the `Authorization` header.

Authentication (`/api/auth`)

Method	Path	Description
POST	`/api/auth/register`	Register a new user (name, email, password)
POST	`/api/auth/login`	Authenticate with email and password; returns JWT
POST	`/api/auth/google`	Authenticate with a Google ID token
GET	`/api/auth/me`	Return the authenticated user's profile
PATCH	`/api/auth/me`	Update name or email
POST	`/api/auth/change-password`	Change password for authenticated user
POST	`/api/auth/profile/image`	Upload a profile image (JPEG/PNG/HEIC, resized to 512×512)
POST	`/api/auth/forgot-password`	Send a password reset email
POST	`/api/auth/reset-password`	Consume a reset token and set a new password

61

Documents (`/api/documents`)

Method	Path	Description
POST	`/api/documents/upload`	Upload and process a payslip PDF
GET	`/api/documents`	List all documents for the authenticated user
GET	`/api/documents/payslip-history`	Structured payslip history with year-level statistics
GET	`/api/documents/recent-payslips`	The N most recent completed payslips (limit 1–12)
GET	`/api/documents/ai-insights`	AI-generated insights based on recent payslips
GET	`/api/documents/:id`	Retrieve a single document
GET	`/api/documents/:id/download`	Download the original PDF
GET	`/api/documents/:id/digest`	Return the AI-generated digest for a completed document
POST	`/api/documents/:id/reprocess`	Re-run extraction on an existing document
POST	`/api/documents/:id/unlock`	Supply a PDF password and re-process
PATCH	`/api/documents/:id/fields`	Manually fill fields the OCR did not extract
DELETE	`/api/documents/:id`	Delete a document and its file
DELETE	`/api/documents/all`	Delete all documents for the authenticated user

Findings (`/api/findings`)

Method	Path	Description
GET	`/api/findings`	Generate and return all financial findings
POST	`/api/findings/savings-forecast`	Compute a savings forecast for two scenarios

62

AI Assistant (`/api/ai`)

Method	Path	Description
POST	`/api/ai/chat`	Submit a chat message; returns a complete response
POST	`/api/ai/chat/stream`	Submit a chat message; streams tokens via SSE
GET	`/api/ai/chat/history`	Retrieve conversation history
GET	`/api/ai/chat/conversations`	List distinct conversation threads
GET	`/api/ai/financial-tips`	Return rule-based financial tips
POST	`/api/ai/full-analysis`	Run the full multi-agent financial analysis

Pension (`/api/pension`)

Method	Path	Description
GET	`/api/pension/analysis`	Full pension analysis for the user
GET	`/api/pension/import-history`	Pension import history
POST	`/api/pension/simulate`	What-if pension scenario simulation
POST	`/api/pension/upload`	Upload pension statement metadata
POST	`/api/pension/upload-file`	Upload pension statement file
GET	`/api/pension/funds`	List user's pension funds
PATCH	`/api/pension/funds/:id`	Update a pension fund record
DELETE	`/api/pension/funds/:id`	Delete a pension fund
GET	`/api/pension/risk-advice`	Risk profile advice
GET	`/api/pension/fund-advice`	Fund-level advice
GET	`/api/pension/leading-funds`	Market leading funds reference
GET	`/api/pension/fund/:id`	Single fund detail

63

Copilot (`/api/copilot`)

Method	Path	Description
GET	`/api/copilot/analysis`	Consolidated financial snapshot
GET	`/api/copilot/problems`	Detected financial problems
PUT	`/api/copilot/profile`	Update copilot financial profile
POST/PUT	`/api/copilot/goals`	Create or update savings goals
DELETE	`/api/copilot/goals/:id`	Delete a goal
POST	`/api/copilot/monthly-report`	Generate monthly markdown report

Insurance (`/api/insurance`)

Method	Path	Description
GET	`/api/insurance`	Retrieve the user's insurance analysis
POST	`/api/insurance/profile`	Save insurance profile data

64

Additional Routes

Method	Path	Description
GET	`/api/health`	Health check (unauthenticated)
GET/POST/PATCH	`/api/onboarding/*`	User onboarding flow
GET/POST/PATCH	`/api/profile/*`	Extended user profile management
GET/POST	`/api/integrations/gmail/*`	Gmail OAuth and payslip import
GET	`/api/financial-health`	Financial health score and breakdown
GET/POST	`/api/notifications/*`	In-app notification management
GET/POST	`/api/recommendations/*`	Personalized recommendation management
GET/POST	`/api/insights/*`	Financial insight records
GET	`/api/tax-assistant/*`	Tax assistant and Form 106 support
GET/POST	`/api/copilot/*`	Copilot assistant endpoints
GET/POST	`/api/score-agent/*`	Score agent endpoints
GET/POST	`/api/dashboard/*`	Dashboard aggregation endpoints
GET/POST	`/api/agents/*`	Agent orchestration endpoints
GET/POST	`/api/summary-email/*`	Summary email delivery

65

Appendix B: Project Setup and Reproduction

B.1 Prerequisites

Node.js 20.x and npm

MongoDB (local, Atlas, or Docker Compose via `npm run dev:docker`)

For OCR on macOS without Docker: `tesseract` (with `heb` pack) and `poppler` (`pdftotext`, `pdftoppm`)

Google Chrome (for `npm run build` in `Final_Project_Book/`)

B.2 Installation

```
bash
# From repository root
npm run install:all
```

B.3 Environment variables (backend `.env`)

Variable	Required	Purpose
`JWT_SECRET`	Yes (≥10 chars)	JWT signing
`MONGODB_URI`	Yes	MongoDB connection
`PORT`	No (default 5000)	API port
`CLIENT_URL`	Yes	CORS whitelist
`GOOGLE_CLIENT_ID`	For OAuth	Google Sign-In
`ANTHROPIC_API_KEY`	For Claude	AI assistant
`OLLAMA_URL`, `OLLAMA_MODEL`	Optional	LLM fallback
`MAX_UPLOAD_SIZE_MB`	No (default 10)	Upload limit

Frontend `.env`: `VITE_API_URL`, `VITE_GOOGLE_CLIENT_ID`.

B.4 Run development stack

```
bash
npm run dev              # backend :5000 + frontend :5173
npm run dev:docker       # MongoDB + backend with OCR in Docker
```


66

B.5 Evaluation and benchmark commands

```
bash
cd backend
npm run eval:ocr
npm run eval:findings
npm run eval:ai-routing
npm run bench:upload-latency
npm test
```

B.6 Build project book PDF

```
bash
cd Final_Project_Book
export                                CHROME_PATH="/Applications/Google
Chrome.app/Contents/MacOS/Google    Chrome"       #    macOS;    updates
figures/puppeteer.config.json
npm run build          # figures + PDF
npm run build:pdf      # PDF only (if figures already rendered)
npm run serve          # preview at http://127.0.0.1:8765
```

Appendix C: analysisData v1.9 Field Reference

Top-level fields produced by the extraction pipeline (`schema_version: "1.9"`):

Group	Key fields	Type / notes
`period`	`month` (YYYY-MM)	string
`salary`	`gross_total`, `net_payable`, `components[]`	numbers + line items
`deductions.mandatory`	`total`, `income_tax`, `national_insurance`, `health_insurance`	numbers
`contributions.pension`	`employee`, `employer`, `base_salary_for_pension`, rate percents	numbers
`contributions.study_fund`	`employee`, `employer`, `base_salary_for_study_fund`	numbers
`tax`	`gross_for_income_tax`, `tax_credit_points`, `personal_credit`	numbers
`employment`	`employer_name`, `employment_start_date`, `employment_type`	string / date
`parties`	`employer_name`, `employee_name`, `employee_id`	string
`quality`	`score`, `extraction_path`, `validation.issues`, `warnings`	metadata
`summary`	`date`, derived salary summaries	UI-facing aggregates

Raw OCR text (`raw.rawText`, `raw.ocr_text`) and `quality.debug` are stripped by `documentSerializer.js` before API responses. Frontend mapping: `frontend/src/utils/documentToPayslip.ts`.

End of Final Project Book

68